

JWT Session Expiry

How the app detects an expired or removed login and signs the user out

In short: the backend is the authority on whether a token is valid. The frontend schedules a convenient local timeout, reacts to backend HTTP 401 responses, and notices when the saved login is removed.

What happens in this implementation

1. **Login:** the app saves the JWT and user profile in local storage. The token includes an `exp` claim.
2. **Idle expiry:** the frontend decodes only the expiry timestamp and schedules a logout for that time. Decoding is not treated as proof that a token is valid.
3. **Protected API request:** frontend API calls use `apiFetch`. If a response is HTTP `401 Unauthorized`, it broadcasts `app:auth-expired`.
4. **Backend validation:** the backend verifies and parses the JWT when an endpoint resolves the authenticated user. Missing, malformed, expired, or no-longer-valid user tokens are returned as HTTP 401.
5. **Logout:** the auth provider removes the app's token, user, links, and groups from local storage; clears React auth state; and notifies cache providers to clear their in-memory data.
6. **Redirect:** the protected app layout sees that no token remains and redirects to `/login`.
7. **Manual storage removal:** other tabs detect local-storage changes immediately. The active tab checks storage on focus and when it becomes visible again.

Expired JWT → backend HTTP 401 → `apiFetch` event → clear local session/cache → login page
JWT expiry time reached while idle → session-expired event → same logout flow
token/user removed from storage → storage/focus check → same logout flow

Key design points

- **401 versus 403:** a 401 means the login is missing or invalid and ends the session. A 403 means the user is authenticated but lacks permission; it should not log them out.
- **Backend stays authoritative:** client-side JWT decoding only provides a timer. A modified token is still rejected by backend signature and expiry validation.
- **Only app-owned keys are removed:** logout removes `token`, `user`, `links`, and `groups`, instead of clearing every key on the domain.
- **Stateless JWT caveat:** clearing the token logs this browser out; it does not revoke a copied token on the server. Server-side revocation requires a token blacklist or refresh-token/session store.

How to sanity-test it

A short manual checklist after restarting the backend and frontend

1. Expiry response from the backend

1. Sign in and open a protected page.
2. In browser DevTools, change one character of the JWT value in local storage, then trigger a protected API call (for example, refresh a page's data).
3. Expected: backend returns HTTP 401, frontend clears the session and caches, shows a session-expired message, and returns to login.

A modified signature is a safe way to test the invalid-token path without waiting for the normal token lifetime.

2. Remove the saved login

1. Sign in, then remove `token` and `user` from local storage.
2. Switch to another tab and back, or hide and show the tab. (Storage events are delivered to other tabs; focus/visibility checks cover the active tab.)
3. Expected: the app clears in-memory auth and cached links/groups, then shows the login page.

3. Normal logout still works

1. Sign in and use the app's Logout action.
2. Expected: the same auth and cache keys are removed and protected routes redirect to login.

Files responsible

- `frontend/src/services/apiFetch.js` — detects HTTP 401 responses.
- `frontend/src/context/AuthProvider.jsx` — expiry timer, storage checks, and session cleanup.
- `frontend/src/pages/MainPage.jsx` — protects app pages and redirects signed-out users.
- `backend/.../user/service/UserServiceImpl.java` — translates invalid JWTs into an authentication error.
- `backend/.../exception/handler/GlobalExceptionHandler.java` — returns HTTP 401 for invalid-token errors.

The app JWT lifetime is configured by the backend in `JwtService`. If that lifetime changes, the frontend timer automatically follows the JWT's `exp` claim.